Branch-Predictor Design Exploration on an OOO RISC-V Processor

Wonju Lee and Shivam Kak

Spring 2026

Problem

The riscvooo OOO core has no branch predictor.
Conditional branches resolve in X, squashing F + D on every wrong-path branch – a 2-cycle penalty per mispredict

Proposal

Add a configurable F-stage predictor to riscvooo and compare three direction-predictor algorithms of increasing complexity:

1-bit BHT · 2-bit BHT · two-level (PC⊕GHR)

Running Branch Predictor on FPGA

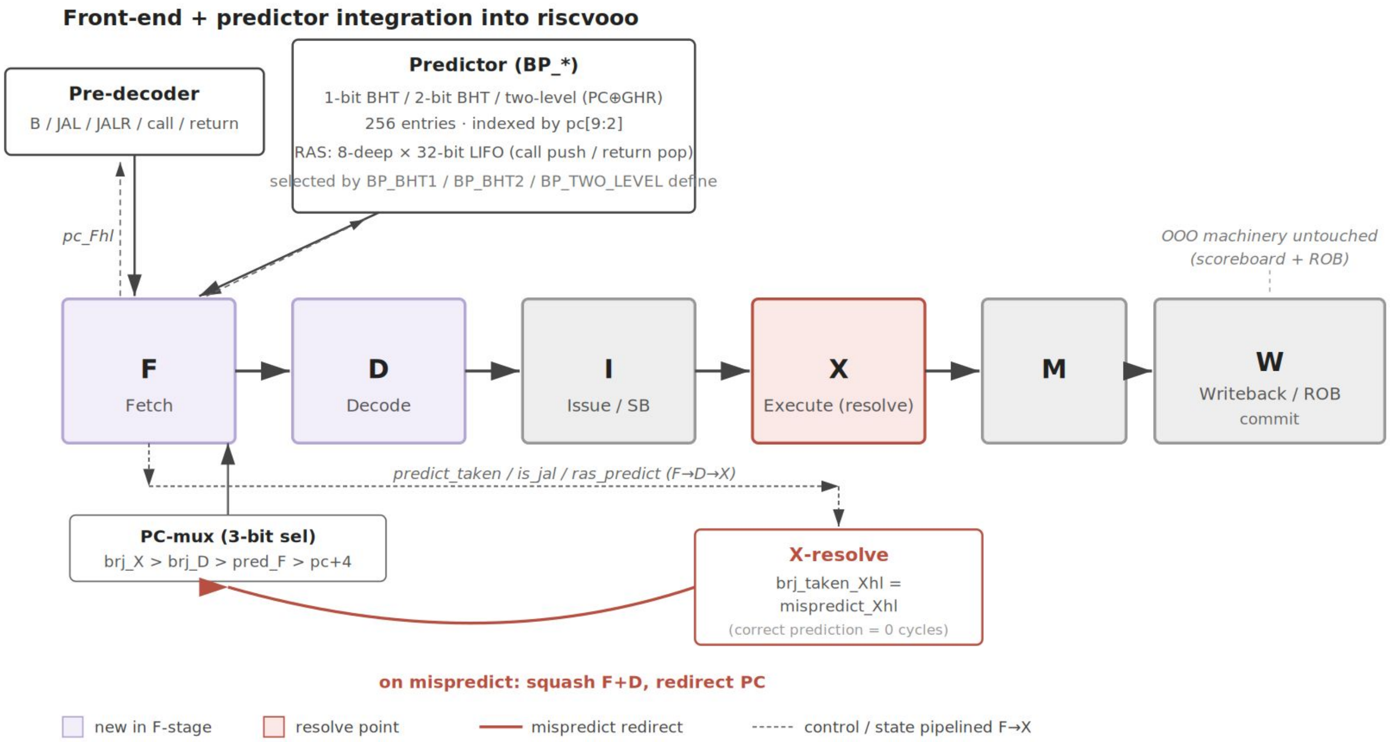
- **Original target:** Nexys-4 DDR / Artix-7 (xc7a100tcsg324-1) on Vivado 2019.1
- **Two-axis goal:** compare predictors on IPC and post-PnR Fmax/utilization

Result: no end-to-end Vivado runs; we built out the flow and ran a yosys-based area proxy

What we built

- **BP sub-package:** pre-decoder + 1-bit BHT, 2-bit BHT, two-level (PC⊕GHR), 8-deep RAS
- **Front-end integration:** predict in F, redirect via new PC-mux input, resolve in X
- **Build-time variant select:** BP_BHT1 / BP_BHT2 / BP_TWO_LEVEL, optional BP_PRED_JAL, BP_RAS
- **OOO architecture:** wrong-path squashed in F+D before issue – ROB never sees it

RTL block diagram



Methodology

- **Lab4 base:** used riscvooo from Lab 4 as starting point
- **Baseline:** all 47 asm tests + 4 ubmarks match lab4 *-ooo.out
- **9-variant sweep:** baseline, static-NT, BHT-1, BHT-2, two-level, +JAL, +full (RAS) for both
- **≈90 s full sweep:** 423 asm runs + 36 ubmark runs PASS on adroit
- **Per-variant counters:** num_branches, num_taken, num_pred_correct, num_mispredicts

Results – IPC across variants

- **Best:** BHT-2 + JAL at 0.799 mean IPC, +7.8%

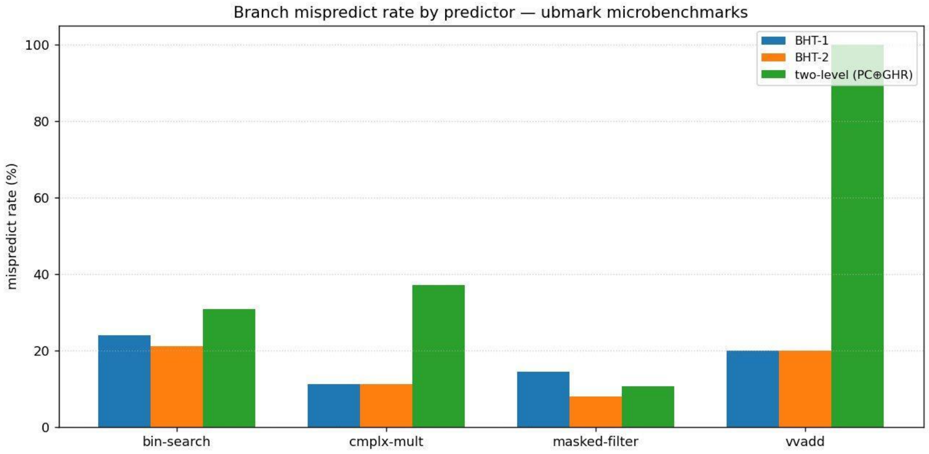
  vs baseline

- **BHT-2 alone:** 0.786 mean (+6.1%)
- **Two-level loses:** 0.770 mean – below BHT-1

  on this workload

Results – mispredict rates

- **BHT-2 best on every benchmark:** masked-filter 7.9% vs BHT-1's 14.4%
- **Two-level pathological on vvadd:** 100% – every one of 10 kernel branches mispredicted
- **Cause:** GHR pollution – PC⊕GHR remaps the same PC across visits



JAL prediction + RAS observations

- JAL prediction - +3.4% IPC on bin-search and on masked-filter
- F-stage uses pc + sext(imm_uj); D-stage redirect suppressed when pred matches
- 8-deep RAS is correct - BHT-2 + JAL and BHT-2 full produce identical numbers on every ubmark
- test_stats_on/off regions contain at most a couple of JALR returns
- Verified in waveform - push on JAL-with-rd=x1, pop on JALR-with-rs1=x1, target match gates D-redirect

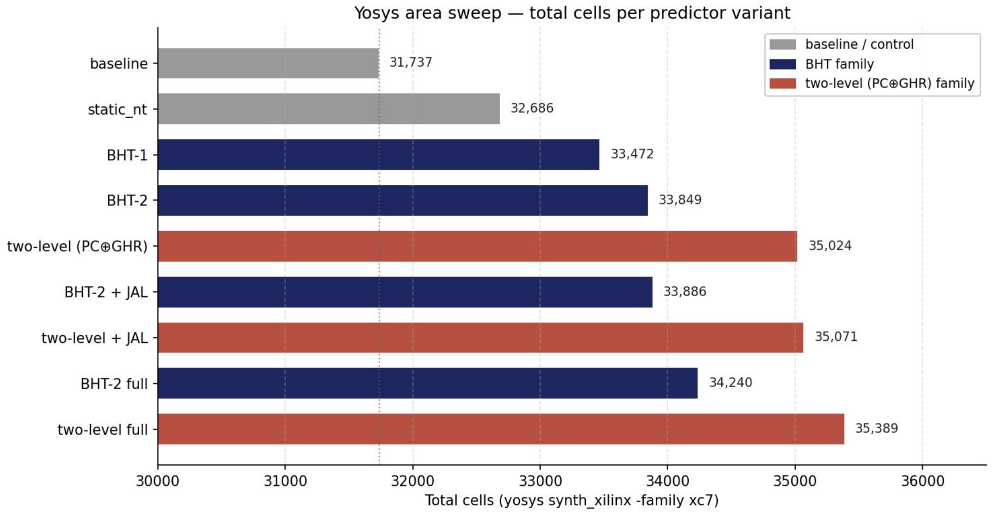
FPGA Setup (Vivado + Yosys)

Vivado Synthesis Data Flow

- **Entry point:** vivado -mode batch -source synth_variant.tcl -tclargs <variant>
- **RTL overlay:** drops per-variant Core/CoreCtrl/CoreDpath into riscvlong/, plus new bp/ for predictor modules
- **Pipeline:** reset_run synth_1 → launch_runs synth_1 → launch_runs impl_1 -to_step write_bitstream
- **Reports:** report_utilization, report_timing_summary, report_timing, report_power → results/<variant>/
- **Post-process:** scp back to laptop; parse_vivado_reports.py builds vivado_summary.tsv (LUTs, FFs, BRAM, DSP, period, WNS, Fmax, power)

Yosys Area Proxy

- **Tool:** yosys 0.64, synth_xilinx -family xc7
- **Predictor adds 5–11%** to total cells over baseline
- **FF deltas match the on-paper budget:** 256×2 PHT for BHT-2; +260 FFs for the 8×32 RAS
- **JAL prediction:** essentially free (~30 cells over matched non-JAL build)
- **Two-level extra LUTs:** from yosys' LUT4/5/6 fan-in tree; Vivado would fold these into LUT-RAM



Discussion

- **GHR pollution** is root cause
  - Loop branches are nearly single-direction, so the GHR cycles through patterns that have no useful correlation
  - PC⊕GHR remaps the same PC across visits – each PHT counter sees only a handful of training cycles before it gets remapped
- **BHT-2 hysteresis wins** where two-level can't
  - Per-PC counters accumulate hysteresis across all visits to the same branch
  - On masked-filter, BHT-2 cuts mispredicts almost in half (96 → 53) by absorbing rare anomalous outcomes
- Global-history correlation pays only when correlation exists
  - These ubmark kernels have none – a more branchy workload could flip the conclusion

Conclusion

- BHT-2 + JAL wins: 0.799 mean IPC, +7.8% over baseline; BHT-1 close behind at 0.782
- Two-level underperforms here because more state, no correlation to exploit on these kernels
- RAS in place for future workloads - allows for deeper recursion / more JALR returns will exercise it
- **Source:** **github.com/Shivamkak19/fpga_riscv_branch_predictor**

Next Steps

- **Run full Vivado sweep:** 9 variants × ≈10–15 min each – single overnight Artix-7 run
    - Re-target overlay paths from riscvlong/ to riscvooo/ (4 patched files)
    - Pull utilization, timing, power reports back via scp; populate vivado_summary.tsv
- **Layer in on-board IPS measurement:** program each variant’s bitstream via Vivado Hardware Manager
    - Combine reported Fmax with cycle counts already in our IPC tables
    - IPS = retired_inst × Fmax / cycles per benchmark per variant
- **Validate the area–Fmax tradeoff:** expect two-level’s extra XOR fan-in to cost a few MHz
    - Hypothesis: BHT-2 + JAL dominates on both axes for this workload